

# Python functions

[https://bioinfmsc8.bio.ed.ac.uk/AY25\\_BPSM15.html](https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM15.html)

---

## Quick links:

Click on any [🏠](#) to get back here!

- [Writing a function](#)
- [First example of a function](#)
- [How to write a function](#)
- [Important things to note \(more\)](#)
- ["Calling" a function](#)
- [Encapsulation](#)
- [Encapsulation "breakers" #1](#), including the None type
- [Encapsulation "breakers" #2](#)
- [Two or more arguments](#)
- [Default values](#) for function arguments
- [Keyword arguments](#)

- [Checking the function: assertions](#)
- [A strange problem](#) with assertions...

---

### Exercises

1. [Amino acid percentages, part one](#)
2. [Amino acid percentages, part two](#)
3. [Base counter](#)
4. [Kmer counting \(again!\)](#)
5. [Kmer counting \(and yet again!\)](#)

- 
- [important git/GitHub stuff at the end](#)

---

## 📌 Writing functions

### Let's write a simple function

Remember our old favourite from before: AT content of a sequence?

```
# Start with a text string that is our DNA sequence
```

```
my_dna = "actgatacatatatatcgatgcgttcat"
```

```
# How long is it?
```

```
length = len(my_dna)
```

```
# How many A bases?
```

```
a_count = my_dna.upper().count('A')
```

```
# How many T bases?
```

```
t_count = my_dna.upper().count('T')
```

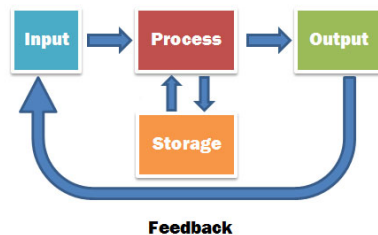
```
# What is the AT proportion?
```

```
at_content = (a_count + t_count) / length
```

```
# Print out the info
```

```
print("AT content of " + my_dna + " is " + str(at_content))
```

```
AT content of actgatacatatatatcgatgcgttcat is 0.6785714285714286
```



If we break it down into bits:

**INPUT bit** was 1 line of code : this can be different each time

**PROCESS bit** was 4 lines of code : this will stay the same, whatever the input sequence is

**OUTPUT bit** was 1 line of code : this can be different each time

We can make this bit of reuseable code into what is called a **function**.

Every time that we need that bit of processing all we have to do is "call" the function.

## 📌 Our first function

**# define a new function that will get the AT content of the DNA**

**# we decide to give the function a name: get\_at\_content**

**# the function only needs a DNA sequence as an input**

**# the only thing that comes out of a function is what we tell it to return**

**#**

**# Our AT content function**

```
def get_at_content(some_dna):  
    length = len(some_dna)  
    a_count = some_dna.upper().count('A')  
    t_count = some_dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return at_content
```

**# Do the internal variables exist before we actually use it?**

**a\_count**

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'a_count' is not defined
```

**some\_dna**

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'some_dna' is not defined
```

**# Call the function to process our input**

**get\_at\_content()**

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: get_at_content() missing 1 required positional argument: 'some_dna'
```

**# Try again!**

**get\_at\_content(some\_dna)**

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'some_dna' is not defined
```

**# Try again, see if these work!**

**my\_dna**

**'actgatacatatatatcgatgcgttcat'**

**get\_at\_content('actgatacatatatatcgatgcgttcat')**

**0.6785714285714286**

**get\_at\_content(my\_dna)**

**0.6785714285714286**

**# Try again, store the returned output in a variable!**

**at = get\_at\_content("actgatacatatatatcgatgcgttcat") # single OR double  
quotes OK here**

**print(at)**

**0.678571428571**

**# Do the internal variables exist now?**

**a\_count**

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'a_count' is not defined
```

**some\_dna**

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'some_dna' is not defined
```

---

## 📌 Important things to note

```
def some_function_name(argument):  
    somevariable1 = action_one(argument)  
    somevariable2 = action_two(argument)  
    somevariable3 = action_three(argument)  
    return someresult
```

- the definition line starts with `def`
- next comes the name of our function (could be anything, but something related to what it does is usually best!)
- function argument names go in round `( )` brackets (argument names are somewhat arbitrary, but it is good to make them something that you understand)
- the `def` line ends with a colon `:`
- the body is indented
- we use the argument names inside the body
- function body ends with a `return` line that is used to return the result: no `return` line means no result!
- we can have several `return` lines in our function, **BUT** whenever a `return` line of code is actually used by the function, the processing then comes out of the function, similar to the `break` command
- defining a function doesn't cause it to run; that only happens when we "call" it!

## 📌 Important things to note (yes, more of them...)

1. Argument variables only exist inside the function
2. When we **write** a function we don't have to worry about how it will be used
  - we just need to know the inputs (arguments) and output that we want to have
3. When we **use** a function we don't have to worry about how it works inside
  - we just need to know what input(s) to give it, and what output we will get back

---

## 📌 "Calling" our new function

Once we've written our function, we can use it in lots of different ways:

```
# Calculate the AT content of a sequence in a file we have in our working directory
```

```
dna2 = open('dna2.txt').read()
```

```
at = get_at_content(dna2)
```

```
# Calculate the AT content of a sequence in a file
```

```
at = get_at_content(open('dna2.txt').read())
```

```
# Print the AT content for a given sequence
```

```
print(get_at_content("ATtaGCGTaGC"))
```

```
# Write the AT content for a sequence to a file
```

```
result = open('output.txt', 'w')
```

```
result.write(str(get_at_content('ACTGTcGa')))
```

```
result.close()
```

This separation of code is very valuable: it is called **"encapsulation"**, in other words "in a capsule".

```
# Write the AT content for a sequence to a file
```

```
with open('output.txt', 'w') as result :  
    result.write(str(get_at_content('ACTGTcGa')))
```

```
3
```

```
result.closed
```



**True**

**print(open("output.txt").read())**

**0.5**

## 📌 Things to try to avoid when writing functions: encapsulation "breakers" #1

# DON'T write a function that relies on variables created outside of the function

# As we would have to know what variables are set in order to write the function.

```
dna = "ATCGCTAGCTGC"
```

# This DIFFERENT function will ONLY work if the variable `dna` already exists!  
# Do NOT do this!

```
def get_at_content():  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return at_content
```

# This works without giving it any arguments because `dna` exists as a variable with a value

```
get_at_content()
```

```
0.4166666666666667
```

# Clear the variable, and what happens to our function output?

```
dna
```

```
'ATCGCTAGCTGC'
```

```
dna = ""
```

```
dna
```

```
''
```

```
get_at_content()
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
  File "<stdin>", line 5, in get_at_content  
ZeroDivisionError: division by zero
```

```
dna=None
```

```
dna
```

```
get_at_content()
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "<stdin>", line 2, in get_at_content  
TypeError: object of type 'NoneType' has no len()
```

```
type(dna)
```

```
<class 'NoneType'>
```

**Huh?!**

The `None` keyword is used to define a null value, or no value at all.

`None` is **not** the same as 0, False, or an empty string.

`None` is a datatype of its own (NoneType) and only `None` can be None.

**# Delete the variable, and what happens to our function output?**

```
del dna
```

```
get_at_content()
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "<stdin>", line 2, in get_at_content  
NameError: name 'dna' is not defined
```

## 📌 Things to try to avoid when writing functions: encapsulation "breakers" #2

**# DON'T write a function that only prints the value instead of returning it**

**# Then we'd have to know how the function will be used in order to write it**

```
def get_at_content(dna):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    print(at_content)
```

```
get_at_content("ATGCGTAtttGAGCAA")
```

0.625

**# Can't I just put the output in a variable?**

```
foo=get_at_content("ATGCGTAtttGAGCAA")
```

0.625

foo

**# Empty string provided: it tries to work**

```
get_at_content("")
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    File "<stdin>", line 5, in get_at_content  
ZeroDivisionError: division by zero
```

**# Forgot to give it an argument... ooops, it doesn't work**

```
get_at_content()
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: get_at_content() missing 1 required positional argument: 'dna'
```

By all means include print statements when checking how/whether your function

is working, but ideally the final version of your functions should only "return" a value.

A good rule of thumb: information gets in by **arguments**, information gets out by **return** value.

## 📌 Improving our function: adding another argument

One problem currently is that we are often getting way too many decimal places in our answer.

### # Our AT content function (version 1)

```
def get_at_content(dna):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return(at_content)
```

### # Try it

```
get_at_content("ATGCGTAtttTTGAGCA")
```

0.6470588235294118

### # Round the AT content to 5 decimal places

```
round(get_at_content("ATGCGTAtttTTGAGCA"),5)
```

0.64706

We can solve this problem by calling the round() function on the answer before we return it.

### # Our AT content function, v2, two decimal places

```
def get_at_content_v2(dna):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return round(at_content, 2)
```

```
get_at_content_v2("ATGCGTAtttTTGAGCA")
```

0.65



## Adding an argument to the function

What if we might want more/fewer decimal places? Just make the argument to `round()` an argument of our function!

### # Our AT content function, v3, specify number of significant figures

```
def get_at_content_v3(dna, sig_figs):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return round(at_content, sig_figs)
```

### # Try the function out, requesting 2 significant figures

```
get_at_content_v3("ATGCGTATtttTGAGCA", 2)
```

0.65

### # Try the function out, requesting 4 significant figures

```
get_at_content_v3("ATGCGTATtttTGAGCA", 4)
```

0.6471

### # What happens if you forget to put in how many significant figures you want

```
get_at_content_v3("ATGCGTATtttTGAGCA")
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: get_at_content_v3() missing 1 required positional argument: 'sig_figs'
```

### # We could `try` it first!

```
try :  
    get_at_content_v3("ATGCGTATtttTGAGCA")  
except :  
    f'Nah, ya broke it, numpty!'
```

Nah, ya broke it, **numpty**!





---

## 📌 Default values for function arguments

In many cases, we don't really care about how many significant figures there are, but in most cases, 2 is probably OK. We can thus add a default to the definition.

**# Our AT content function, v4, specify number of signif. figures, have default of 2**

```
def get_at_content_v4(dna, sig_figs=2):  
    length = len(dna)  
    a_count = dna.upper().count('A')  
    t_count = dna.upper().count('T')  
    at_content = (a_count + t_count) / length  
    return round(at_content, sig_figs)
```

**# Try the function out, requesting 4 significant figures**

```
get_at_content_v4("ATGCGTATtttTGAGCA", 4)
```

0.6471

**# Try the function out, not specifying number of significant figures required**

```
get_at_content_v4("ATGCGTATtttTGAGCA")
```

0.65

## 📌 Keyword arguments

In all the examples above, we supply the arguments in the same order as the definition.

If we want to use a different order (or just be more explicit) we can use keyword arguments

```
# Try the function out, requesting 4 significant figures, but we had  
# forgotten what the argument order should be (and got it wrong!)
```

```
get_at_content_v4(4,"ATGCGTATtttTGAGCA")
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
  File "<stdin>", line 2, in get_at_content_v4  
TypeError: object of type 'int' has no len()
```

```
# Say what we want this time: order that we provide is no longer important,  
# as we are explicitly saying which is which
```

```
get_at_content_v4(sig_figs=4, dna="ATGCGTATtttTGAGCA")
```

```
0.6471
```

```
get_at_content_v4(dna="ATGCGTATtttTGAGCA",sig_figs=4)
```

```
0.6471
```

```
get_at_content_v4(dna=my_other_dna,sig_figs=4)
```

```
0.3786
```

```
# Still doesn't help with this situation...!
```

```
get_at_content_v4(dna="ATGCGTATtttTGAGCANNNNNNNNNNNNNNNNNNNN")
```

```
0.22
```

## 📌 Testing functions

We have written a new function, so we must test that it is working correctly.

```
get_at_content_v4("A")
```

```
1.0
```

```
get_at_content_v4("t")
```

```
1.0
```

```
get_at_content_v4("c")
```

```
0.0
```

**# Use a `for` loop to check things, calling our function each time!**

```
for base in 'gacx':  
    print("Was A or T? " + base.upper() + ": " + str(get_at_content_v4(base) == 1))
```

```
Was A or T? G: False  
Was A or T? A: True  
Was A or T? T: True  
Was A or T? C: False  
Was A or T? X: False
```

In Python, we test functions using `assert` with a condition for which we know the answer.

If the assertion is evaluated as being correct, then there is no output: it was "True", so Python doesn't have anything to disagree about.

```
assert get_at_content_v4("A") == 1  
assert get_at_content_v4("AAA") == 1  
assert get_at_content_v4("G") == 0  
assert get_at_content_v4("ATGC") == 0.5
```

**# This IS correct, so no output**

```
assert get_at_content_v4("AAA") == 1
```

**# This ISN'T correct, so we get an assertion error: the statement wasn't True.**

```
assert get_at_content_v4("AAA") == 3
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
AssertionError
```

## **Why should we do assertions?**

Assertions are good for:

- providing some documentation about the behaviour of the function
- reassuring you that your function is giving the right answer
- letting you know if you break the function
- demonstrating how the function can be used to other people

## 📌 A strange problem with assertions...

As you might remember, sometimes Python3 makes slight rounding errors when carrying out operations on floating point numbers:

```
# Some quick maths
```

Two plus two is four, minus one, that's three, quick maths

```
2 + 2 - 1 = 3
```

```
File "<stdin>", line 1
SyntaxError: cannot assign to operator
```

```
2 + 2 - 1
```

```
3
```

Two plus two is four, minus one, that's three, quick maths

```
1 + 2
```

```
3
```

```
1.01 + 2.0
```

```
3.01
```

```
# Some more quick maths: huh?!
```

```
1.01 + 2.01
```

```
3.0199999999999996
```

```
# Some quick evaluations...
```

```
get_at_content_v4("A")
```

```
1.0
```

```
get_at_content_v4("A") == 1.0000000000000001
```

```
False
```

```
get_at_content_v4("A") == 1.0000000000000001
```

True

This can lead to problems when testing assertions:

**# Simple function to add two numbers**

```
def add_two_numbers(a,b):  
    return a + b
```

```
add_two_numbers(1.01,2.01)
```

```
3.0199999999999996
```

```
add_two_numbers(1.01,2.01) == 3.02
```

```
False
```

**# Test the function using an assertion (which will fail despite being True!)**

```
assert add_two_numbers(1.01, 2.01) == 3.02
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
AssertionError
```

**# Really? Try again!**

```
round(add_two_numbers(1.01, 2.01),2)
```

```
3.02
```

**# Deal with this by using round() in the assertion itself**

```
round(add_two_numbers(1.01, 2.01),2) == 3.02
```

```
True
```

```
assert round(add_two_numbers(1.01, 2.01),2) == 3.02
```

**# Or on both sides of the assertion**

```
assert round(add_two_numbers(1.01, 2.01)) == round(3.02)
```

**# Try to deal with this in the function first, using round()**



[Give feedback](#)  
[Ask for help](#)

```
def add_two_numbers_v2(a,b):  
    return round(a + b)
```

**# Now try again with version 2!**

```
add_two_numbers_v2(1.01,2.01)
```

**3**

You have been warned...!



---

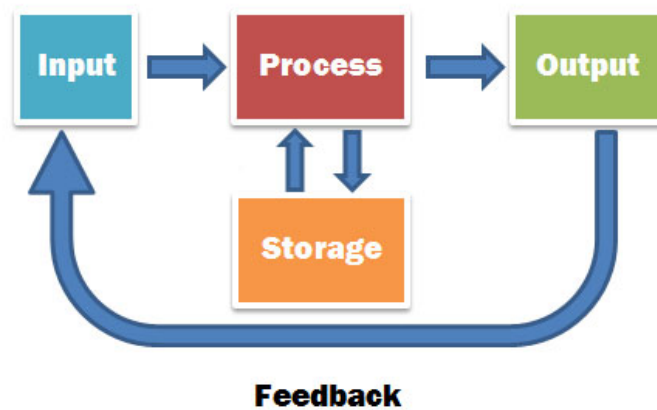
## 🏠 Exercises

### Let's try and be tidy...

Make a directory within your homespace for today's work, and change directory to it.

```
mkdir ~/Exercises/Lecture15
```

```
cd ~/Exercises/Lecture15
```



---

## 📌 Amino acid percentages, part one

Write a Python function that takes two arguments (a protein sequence and an amino acid residue code) and returns the **percentage** of the protein that the amino acid makes up.

Once you have written a function, use the following assertions to test your function:

- `assert round(some_function_name("MSRSLLLRFLFLLLLPPLP", "M")) == round(5)`
- `assert round(some_function_name("MSRSLLLRFLFLLLLPPLP", "r")) == round(10)`
- `assert round(some_function_name("MSRSLLLRFLFLLLLPPLP", "L")) == round(50)`
- `assert round(some_function_name("MSRSLLLRFLFLLLLPPLP", "Y")) == round(0)`

Remember to change the name of the function in these assert statements to match your function name!

---

## 📌 Amino acid percentages, part two

Modify the function from the previous exercise above so that it accepts a list of amino acid residues rather than a single one, and count these within the protein sequence.

If no list is given, the function should return the percentage of hydrophobic amino acid residues (i.e. amino acids A, I, L, M, F, W, Y, V).

Your function should pass the following assertions:

- `assert round(some_other_function_name("MSRSLLLRFLFLLLLPPLP", ["M"])) == 5`

- `assert round(some_other_function_name("MSRSLLLRFLFLLLLPPPLP", ['F', 'S', 'L'])) == 70`
- `assert round(some_other_function_name("MSRSLLLRFLFLLLLPPPLP")) == 65`

**Hint:** To get this one to work, you'll have to go through the list of amino acid residues one at a time, generate the count for each one, and come up with a total count.

---

### 📌 Base counter

Write a Python function that will take a DNA sequence along with an optional threshold and return True or False to indicate whether the DNA sequence contains a high proportion of undetermined bases (i.e. **not** A, T, G or C).

Write some assertions to test whether the function works.

---

### Kmer counting (again!)

K-mers are short DNA subsequences with length "k" bases, and are usually generated using the sliding window method we used in previous lecture exercises.

Write a Python function that, given any DNA sequence, will print all the k-mers (e.g. 4-mers) that occur more than n times.

E.g. with `dna="ATGCATCATG"`, `kmersize=2` and `minfrequency=2` will output

**AT**

because the kmers (kmersize) are 2 bases long, and there are three (minfrequency was 2) instances of "AT"

Code you wrote for the previous lecture's exercises could be useful here...

---

### 📌 Kmer counting (and yet again!)

Write a Python script/programme that, given any DNA sequence, will print a list of k-mers that occur more than some number of times.

The user should be asked to supply, on the command line, the following:

1. the sequence of interest
2. the kmer length for analysis
3. the threshold frequency of kmers found (i.e. the "more than this number" value)

---

---





---

## **POSSIBLE ANSWERS**

---

**Python / Linux file manipulations first!**



```
mkdir ~/Exercises/Lecture15
```

```
cd ~/Exercises/Lecture15
```

## Amino acid percentages, part one

Write a Python function that takes two arguments (a protein sequence and an amino acid residue code) and returns the **percentage** of the protein that the amino acid makes up.

This is pretty similar to the AT counting code. We can start off with a specific protein sequence and amino acid residue, probably best to use the one in the assertions we have been asked to do as a test set.

### # INPUT

```
protein = "MSRSLLLRFLFLLLLPPLP"  
aa = "R"
```

### # PROCESS

```
aa_count = protein.count(aa)  
protein_length = len(protein)  
percentage = aa_count * 100 / protein_length
```

### # OUTPUT

```
print(percentage)
```

10.0

Now we just turn the above little programme into a function: protein sequence and aa variables are arguments, and the percentage variable is the return value.

### # AA percentage function, made it case insensitive

```
def get_aa_percentage(protein, aa):  
    aa_count = protein.upper().count(aa.upper())  
    protein_length = len(protein)  
    percentage = (aa_count / protein_length) * 100  
    return percentage
```

### # Do some checks for upper/lower case

```
get_aa_percentage("MSRSLLLRFLFLLLLPPLP", "R")
```

10.0

```
get_aa_percentage("MSRSLLLRFLFLLLLPPLP", "r")
```

10.0

```
get_aa_percentage( "MSrSLLlRfLLfLLlLPPLP", "r")
```

```
10.0
```

```
get_aa_percentage( "MSrSLLlRfLLfLLlLPPLP", "R")
```

```
10.0
```

### # Test it with the supplied assertions

```
assert get_aa_percentage("MSRSLLLRfLLfLLlLPPLP", "M") == 5
assert get_aa_percentage("MSRSLLLRfLLfLLlLPPLP", "r") == 10
assert get_aa_percentage("msrsl11rf11f111lpplp", "L") == 50
assert get_aa_percentage("MSRSLLLRfLLfLLlLPPLP", "Y") == 0
```

**# No outputs, so they must be TRUE, so ALL OK!**

## Amino acid percentages, part two

Modify the function from part one so that it accepts a list of amino acid residues rather than a single one, and count these within the protein sequence.

If no list is given, the function should return the percentage of hydrophobic amino acid residues (A, I, L, M, F, W, Y and V).

The new thing here is that we need to write a function that takes a list as one of its arguments.

Let's take the same approach as for part one - pick a specific input and write the code to solve it, then turn it into a function.

We will count the number of each amino acid in the list in turn and use a variable to keep a running total of the counts.

### # INPUT

```
protein = "MSRSLLLRfLLfLLlLPPLP"
aa_list = ['M', 'L', 'F']
```

### # PROCESS

**# the counter variable will hold the total number of matching residues**

```
counter = 0
```

**# for loop to go through the list**

```
for aa in aa_list:
    print("counting number of " + aa)
    aa_count = protein.upper().count(aa.upper())
```

## # add the number for this residue to the total count

```
counter = counter + aa_count
print("running total is " + str(counter))
```

## # OUTPUT

```
percentage = (counter / len(protein)) * 100
print("final percentage is " + str(percentage))
```

## # Here was the output

```
counting number of M
running total is 1
counting number of L
running total is 11
counting number of F
running total is 13
final percentage is 65.0
```

## # This looks OK, time to turn it into a function

### # Our revised function, case insensitive

```
def get_aa_percentage_v2(protein, aa_list):
    protein_length = len(protein)
    counter = 0
    for aa in aa_list:
        aa_count = protein.upper().count(aa.upper())
        counter = counter + aa_count
    percentage = (counter / protein_length) * 100
    return percentage
```

### # Quick test of our revised function

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPPLP", ['M', 'L', 'F'])
```

```
65.0
```

### # On a `for` loop starting line, a string automatically becomes a list...

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPPLP", 'mlf')
```

```
65.0
```

## # Now test with the assertions

```
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPPLP", ["M"]) == 5
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPPLP", ['F', 'S', 'L']) == 70
```

## # Everything passed apart from last one: forgot the default setting...

```
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPPLP") == 65
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: get_aa_percentage_v2() missing 1 required positional argument: 'aa_list'
```

## # Easiest way to do this is to change the function definition line

**# Our revised function, final version**

**# This function calculates the proportion of a list of amino acids as supplied in a string**

**# Written by s123456 on 04 Nov 25**

**#**

```
def get_aa_percentage_v2(protein, aa_list='aILMFYWV'):  
    protein_length = len(protein)  
    counter = 0  
    for aa in aa_list:  
        aa_count = protein.upper().count(aa.upper())  
        counter = counter + aa_count  
    percentage = (counter / protein_length) * 100  
    return percentage
```

**# Now test with the assertions**

```
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP", ["M"]) == 5  
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP", ['F', 'S', 'L']) == 70  
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP", 'FSL') == 70  
assert get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP") == 65
```

**# No outputs, so must ALL be OK!?**

**# err.... no! breaking the function....way too easy!**

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP","I1234")
```

50.0

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP","lab1234")
```

50.0

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP","lablab1234")
```

100.0

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP","lablablab1234")
```

150.0

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP",  
["I","1","2","I","3","4","I"])
```

150.0

```
get_aa_percentage_v2("MSRSLLLRFLFLLLLPPLP",set(["I","1","2","I","3","4","I"]))
```

50.0

Any suggestions for an alternative solution?!



---

## Base counter

Write a function that will take a DNA sequence along with an optional threshold and return True or False to indicate whether the DNA sequence contains a high proportion of undetermined bases (i.e not A, T, G or C).

Write some assertions to test whether the function works.

This time we will jump straight to writing the definition. There are several different ways to measure the **number** of undetermined bases.

### # Function version 1

# add up the number of A, T, G and C then subtract the total from the length of the sequence

```
def count_undetermined_1(dna):  
    total_good_bases = 0  
    for base in ['A', 'T', 'G', 'C']:  
        total_good_bases = total_good_bases + dna.upper().count(base)  
    return len(dna) - total_good_bases
```

### # Quick test

```
count_undetermined_1('atucgtgractanctgactg')
```

3

### # Function version 2

# look at each base in the DNA sequence individually and check if it's undetermined or not

```
def count_undetermined_2(dna):  
    total_undetermined = 0  
    for base in dna.upper():  
        if base not in ['A', 'T', 'G', 'C']:  
            total_undetermined = total_undetermined + 1  
    return total_undetermined
```

## # Quick test

```
count_undetermined_2('atucgtgractanctgactg')
```

3

## # Function version 3

### # See whats left if we remove all expected bases

```
def count_undetermined_3(dna):  
    total_undetermined = len(dna.upper().replace('A','').replace('T','').replace('G','')).r  
    return total_undetermined
```

## # Quick test

```
count_undetermined_3('atucgtgractanctgactg')
```

3

Let's go with this third way. We also need to convert the number to a proportion.

## # Function version 4

### # Remove all expecteds, return the proportion rather than the number

```
def count_undetermined_4(dna):  
    total_undetermined = len(dna.upper().replace('A','').replace('T','').replace('G','')).r  
    prop_undetermined = total_undetermined / len(dna)  
    return prop_undetermined
```

## # Quick test

```
count_undetermined_4('atucgtgractanctgactg')
```

0.15

## # Function version 5

### # Remove all expecteds, add a threshold argument with a default,

### # return boolean (True or False) if above/below the threshold

```
def count_undetermined_5(dna, threshold=0.1):  
    total_undetermined = len(dna.upper().replace('A','').replace('T','').replace('G','')).r  
    prop_undetermined = total_undetermined / len(dna)
```

### # What is returned is boolean: either True or False

```
    return prop_undetermined >= threshold
```

## # Quick tests

```
count_undetermined_5('atucgtgractanctgactg')
```

**True**

```
count_undetermined_5('atucgtgractanctgactg', 0.1)
```

**True**

```
count_undetermined_5('atucgtgractanctgactg', 0.2)
```

**False**

**# How about this as a "better"/different way...?**

```
list( set.difference(set(dna.lower()),set('gatc')) )
```

```
['n', 'u', 'r']
```

```
for e in list(set.difference(set(dna.lower()),set('gatc'))):  
    print(e.upper(), dna.count(e))
```

```
N 1
```

```
U 1
```

```
R 1
```

etc



## Kmer counting (again....!)

Write a function that, given any DNA sequence, will print all the k-mers (e.g. 4-mers) that occur more than n times.

E.g. with dna="ATGCATCATG", kmer size=2 and min frequency=2 will output  
AT

```
# Write a FUNCTION that, given any DNA sequence, will print
# all the k-mers (of a chosen size e.g. 4-mers) that occur more than some
# chosen number of times.
```

```
# Written by s123456 on 04 Nov 25
```

```
# -----
```

```
# Set some sensible defaults for the function
```

```
def find_my_kmers(dna,ksize=2,min_kfreq=3):
```

```
# Error traps for inappropriate values, what to put
```

```
    if ksize > len(dna) :
        return "Sorry, your kmer length is longer than your DNA (" + str(len(dna)) + " bases) .
    if ksize < 2 or ksize > 50 :
        return "Sorry, inappropriate kmer length, only 2 to 50 accepted here."
    print("Processing sequence of length",len(dna),"for kmers longer than",ksize,"and frequ
    kmersfound = []
    kmerstarts = list(range(0,len(dna)))
    for base in kmerstarts:
        if (base+ksize) < len(dna)+1:
            seqout = (dna)[base:base+ksize]
            kmersfound = kmersfound + [seqout]
    nrset = list(set(kmersfound))
```

```
# Maybe use a list for the multiple returned values?
```

```
    returnstuff = []
    for kfreqfind in nrset:
        if kmersfound.count(kfreqfind) > min_kfreq :
            returnstuff.append(kfreqfind.upper()+" : "+str(kmersfound.count(kfreqfind)))
    return print(returnstuff)
```

**# Let's try it...**

**find\_my\_kmers("ttagatcctgaacgtgaacgcacggatttagatcctgaacgtgaacgcacggat",2,2)**

**Processing sequence of length 54 for kmers longer than 2 and frequency greater than 2**

**['TT: 3', 'AC: 6', 'GA: 8', 'AT: 4', 'AA: 4', 'TG: 4', 'CG: 6']**

**find\_my\_kmers("ttagatcctgaacgtgaacgcacggatttagatcctgaacgtgaacgcacggat")**

**Processing sequence of length 54 for kmers longer than 2 and frequency greater than 3**

**['AC: 6', 'GA: 8', 'AT: 4', 'AA: 4', 'TG: 4', 'CG: 6']**

**find\_my\_kmers("ttagatcctgaacgtgaacgcacggatttagatcctgaacgtgaacgcacggat",55)**

**'Sorry, your kmer length is longer than your DNA (54 bases).'**

**find\_my\_kmers("ttagatcctgaacgtgaacgcacggatttagatcctgaacgtgaacgcacggat",51)**

**'Sorry, inappropriate kmer length, only 2 to 50 accepted here.'**

**find\_my\_kmers("ttagatcctgaacgtgaacgcacggatttagatcctgaacgtgaacgcacggat",50)**

**Processing sequence of length 54 for kmers longer than 50 and frequency greater than 3**

**[]**

## **Kmer counting (...and again!)**

Write a Python script/programme that, given any DNA sequence, will print a list of k-mers that occur more than some number of times.

The user should be asked to supply, on the command line, the following:

1. the sequence of interest
2. the kmer length for analysis
3. the threshold frequency of kmers found (i.e. the "more than this number" value)

I've called this script `kmergetter.py` ...

```
#!/usr/bin/python3
```

```
# shebang on the first line
```

```
# Write a Python script/programme that, given any DNA sequence, will print  
# all the k-mers (of a chosen size e.g. 4-mers) that occur more than some  
# chosen number of times.
```

```
# User interaction to supply all details
```

```
# Written by s123456 on 04 Nov 25
```

```
# -----
```

```
# Module loading
```

```
import os
```

```
# Define the function with defaults where possible
```

```
def new_find_my_kmers(dna, ksize=2, min_kfreq=2) :  
    kmersfound = []  
    kmerstarts = list(range(0, len(dna)))  
    for base in kmerstarts:  
        if (base+ksize) < len(dna)+1:  
            seqout = (dna)[base:base+ksize]  
            kmersfound = kmersfound + [seqout]  
    nrset = list(set(kmersfound))  
    returnstuff = []  
    for kfreqfind in nrset:  
        if kmersfound.count(kfreqfind) > min_kfreq :  
            returnstuff.append(kfreqfind.upper()+" : "+str(kmersfound.count(kfreqfind)))  
    return returnstuff
```

```
# Define the inputs with defaults if user doesnt supply values
```

```
inputdna = input ("What is your sequence?\n\t").upper()  
if inputdna :  
    inputksize = int ( input ("What kmer size shall I use?\n\t") or 2)  
    if (inputksize < 2 or inputksize >= len(inputdna) or inputksize > 50) :  
        inputksize = 2  
        print("Inappropriate value chosen, resetting to 2\n\t")  
    inputmin_kfreq = int ( input ("What minimum frequency shall I use?\n\t") or 2)  
    print("Thanks!  Processing:\n"+inputdna+"\n for a kmersize of "  
        +str(inputksize)+",\n reporting frequencies greater than "  
        +str(inputmin_kfreq)+"\n")
```

```
# Process the input by calling the find_my_kmers function
```

```
outputstuff = new_find_my_kmers(dna=inputdna, ksize=inputksize, min_kfreq=inputmin_kfreq)  
outputstuff.sort()
```

```
# Open the pipe to an output file for our results
```

```
myfilename="kmerouts"+"_KMER"+str(inputksize)+"_MIN"+str(inputmin_kfreq)+".txt"  
outputfilepipe = open(myfilename, "w")
```

```
chmod 755 kmergetter.py
```

**./kmergetter.py**

```
What is your sequence?
      GATATAGTGATGATGATGTAGTGTGATGATGATGATGATGATAG
What kmer size shall I use?

What minimum frequency shall I use?

Thanks!  Processing:
GATATAGTGATGATGATGATGTAGTGTGATGATGATGATGATGATAG
  for a kmersize of 2,
  reporting frequencies greater than 2

Results:

['AG: 3', 'AT: 11', 'GA: 10', 'GT: 4', 'TA: 4', 'TG: 11']

Contents of the output:

### Kmer analysis
#SQ GATATAGTGATGATGATGTAGTGTGATGATGATGATGATGATAG
#KMER 2
#MIN 2
AG: 3
AT: 11
GA: 10
```

GT: 4  
TA: 4  
TG: 11

## ## RUN 2: checking the kmer size error trap

### ./kmergetter.py

```
What is your sequence?
    GATATAGTGATGATGATGTAGTGTGATGATGATGATGATGATGATAG
What kmer size shall I use?
    55
Inappropriate value chosen, resetting to 2

What minimum frequency shall I use?
    5
Thanks! Processing:
GATATAGTGATGATGATGATGTAGTGTGATGATGATGATGATGATGATAG
    for a kmersize of 2,
    reporting frequencies greater than 5
```

Results:

```
['AT: 11', 'GA: 10', 'TG: 11']
```

Contents of the output:

```
### Kmer analysis
#SQ GATATAGTGATGATGATGTAGTGTGATGATGATGATGATGATGATAG
#KMER 2
#MIN 5
AT: 11
GA: 10
TG: 11
```

## ## RUN 3: make it so that it has no kmers for output

### ./kmergetter.py

```
What is your sequence?
    GATATAGTGATGATGATGTAGTGTGATGATGATGATGATGATGATAG
What kmer size shall I use?
    2
What minimum frequency shall I use?
    15
Thanks! Processing:
GATATAGTGATGATGATGATGTAGTGTGATGATGATGATGATGATGATAG
    for a kmersize of 2,
    reporting frequencies greater than 15
```

No kmers met the criteria, so no outputs!

## ## RUN 4: try it with a character where an integer is needed

### ./kmergetter.py

```
What is your sequence?
    GATATAGTGATGATGATGTAGTGTGATGATGATGATGATGATGATAG
What kmer size shall I use?
    x
Traceback (most recent call last):
  File "./kmergetter.py", line 23, in <module>
    inputsize = int ( input ("What kmer size shall I use?\n\t") or 2)
ValueError: invalid literal for int() with base 10: 'x'
```

## ## RUN 5: don't give it any DNA sequence to work on (just press return)

**./kmergetter.py**

What is your sequence?

Sorry, really can't do any of this without a sequence!

## ## RUN 6: give it numbers instead of a DNA sequence

**./kmergetter.py**

What is your sequence?

312331

What kmer size shall I use?

What minimum frequency shall I use?

Thanks! Processing:

312331

for a kmersize of 2,  
reporting frequencies greater than 2

No kmers met the criteria, so no outputs to file!

## ## RUN 7: give it some non-DNA sequence text

**./kmergetter.py**

What is your sequence?

Python is always on my mind: python, python, PYTHON!

What kmer size shall I use?

6

What minimum frequency shall I use?

3

Thanks! Processing:

PYTHON IS ALWAYS ON MY MIND: PYTHON, PYTHON, PYTHON!

for a kmersize of 6,  
reporting frequencies greater than 3

Results:

['PYTHON: 4']

Contents of the output:

### Kmer analysis

#SQ PYTHON IS ALWAYS ON MY MIND: PYTHON, PYTHON, PYTHON!

#KMER 6

#MIN 3

PYTHON: 4

So, still some room for improvement: other error traps to make sure it is DNA sequence, also maybe say what the output filename is, different format for the filename, etc. etc. !?

Possibly also not the best way to do things, as we will see from the alternative offered by the use of `dictionaries`, which we shall cover in the next lecture...!

## Suggestion

As an alternative and very flexible approach, you could have a file with all the functions in called, for example, `myfunctions_v1.py`

In a python3 terminal session, you can access those functions in much the same way as you would import a module, just as we did in the answers to the last lecture's exercises. So, assuming that `myfunctions_v1.py` was in your current working directory, you can import them using the command:

```
import myfunctions_v1
```

and then the functions that you wrote in the `myfunctions_v1.py` file will be available. If you change the `myfunctions_v1.py` file, then just do the import line again:

```
import myfunctions_v1
```

---

## git and GitHub time!

**# Local repository actions (the absolute minimum set of commands required here...)**

### git st

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    deleted:    ../Lecture04/CodeFiles.tar.gz
    modified:   ../Lecture06/run_this_script_v1
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
    ../BN.out
    ../BN.sh
    ../Lecture04/Als_ICA/
    ../Lecture04/all_the_things_I_did
    ../Lecture04/outs/
    ../Lecture05/Country.Mozambique.details
    ../Lecture07/Cosmoscarta.nuc.fa
    ../Lecture07/Cosmoscarta.nuc.gis
    ../Lecture07/myfileofaccessions.txt
    ../Lecture12/AJ223353.fasta
    ../Lecture12/AJ223353.genbank
    ../Lecture12/AJ223353_noheader.fasta
    ../Lecture12/AJ223353_noheader.fasta2
    ../Lecture12/AJ223353_noheader.fasta3
    ../Lecture12/All_exons.fasta
    ../Lecture12/All_noncodings.fasta
    ../Lecture12/local_exon01.fasta
    ../Lecture12/local_exon02.fasta
```

```
../Lecture12/local_intron01.fasta
../Lecture12/my_dna.txt
../Lecture12/plain_genomic_seq.txt
../Lecture12/quick_blast_check.out
../Lecture12/remote_exon01.fasta
../Lecture12/remote_noncoding01.fasta
../Lecture12/remote_noncoding02.fasta
../Lecture12/shutilversion.txt
../Lecture13/100_199/
../Lecture13/200_299/
../Lecture13/300_399/
../Lecture13/400_499/
../Lecture13/500_599/
../Lecture13/600_699/
../Lecture13/700_799/
../Lecture13/800_899/
../Lecture13/900_999/
../Lecture13/Cleaned_sequences.txt
../Lecture13/Cleaned_sequences2.txt
../Lecture13/Cleaned_sequences3.txt
../Lecture13/Exercise2_coding_sequence.fasta
../Lecture13/Lecture13.tar.gz
../Lecture13/dna_files/
../Lecture13/exons.txt
../Lecture13/genomic_dna2.txt
../Lecture13/input.txt
../Lecture13/remote_exon01.fasta
../Lecture14/__pycache__/
../Lecture14/data.csv
./
```

no changes added to commit (use "git add" and/or "git commit -a")

## # You might want all your python scripts?

**git add \*.py**

**git commit -m "Scripts from the python functions lecture"**

```
[master c49df45] Scripts from the python functions lecture
1 file changed, 73 insertions(+)
create mode 100755 Lecture15/kmergetter.py
```

## git remote

```
AllLectures
Lecture07
origin
```

## # What WAS that origin link?!

**git remote get-url origin**

<https://github.com/B123456-2121/MyFirstPipeline.git>

## # What is the AllLectures link?!

**git remote get-url AllLectures**

[https://github.com/B123456-2121/BPSM\\_LectureStuff.git](https://github.com/B123456-2121/BPSM_LectureStuff.git)

## # OK, push our local repository to our Lectures one at GitHub

**git push AllLectures master**



```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 256 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.47 KiB | 754.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
   3b8bf41..c49df45  master -> master
```

## git log

```
c49df45 (HEAD -> master, AllLectures/master) Scripts from the python functions lecture
3b8bf41 My scripts from the python conditionals exercises Py04
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbc5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

**All done, all good!**

---

## Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!



Sources used